JSD TATA EMERGENCY SERVICES

System Design

Requirements, design decisions, scale, resilience and cost

A layered design document: an executive overview of the platform followed by detailed component design, the dispatch algorithm, the security model, and non-functional treatment of scalability, resilience and cost.

Document	System Design
Date	25 June 2026
Audience	Executive summary + engineering detail (layered)

Status

1. Executive summary

The platform provides on-demand emergency response coordination for the Tata Steel Jamshedpur township across three services — ambulance, fire and blood logistics — with multi-channel intake (web portal, machine API, and a voice agent). It is built serverless-first for elasticity and low operational overhead, and integrates AI (Amazon Bedrock) for natural-language intake and policy management.

2. Goals & scope

- Dispatch the nearest appropriate unit automatically, with manual override for dispatchers.
- Give every requester a traffic-aware ETA and a live tracking link.
- Let hospitals/blood banks integrate machine-to-machine with scoped, least-privilege access.
- Adapt operational policy from a plain-language document, without redeploying code.
- Scale from demo to township load without server management.

3. Requirements

3.1 Functional

- Create/auto-dispatch emergencies (medical, fire, blood); mass-casualty multi-unit dispatch.
- Nearest-unit assignment by zone proximity; hospital selection by specialty and capability.
- Manual reassignment of unit/hospital; cancel and complete.
- Traffic-aware ETA; live public tracking; email/SMS notifications.
- Admin policy upload; analytics dashboard.

3.2 Non-functional

- Availability: serverless, multi-AZ managed services; graceful degradation when externals fail.
- Security: SSO JWT verification, scoped API keys, CORS lockdown, throttling, input validation, log hygiene.
- Performance: dispatch decision in well under a second; ETA enriched via OSRM with caching.
- Cost: pay-per-use; near-zero idle cost.

4. High-level architecture

Clients reach a CloudFront-delivered SPA and an API Gateway HTTP API. A single Lambda holds the domain logic and talks to DynamoDB and the external/AI services. A second Lambda performs policy synchronisation. See the master architecture diagram on the next page.

Master architecture

All services and how requests flow from clients to data and external systems.

5. Key design decisions & trade-offs

Decision	Rationale	Trade-off
Single Lambda (modular monolith)	Simple deploy, shared code, low cold-start surface.	Less independent scaling per route.
DynamoDB single-digit-ms KV	Elastic, serverless, predictable latency.	Access patterns must be designed up front (GSIs).
JWT verified in-Lambda	No extra authorizer infra; full control.	Must manage JWKS caching/ rotation in code.
OSRM public routing	Free road ETAs for the POC.	No live traffic feed; congestion simulated.
Policy-as-config via Bedrock	Non-engineers tune behaviour from a PDF.	Relies on LLM extraction + sanitisation.

6. Component design

6.1 Frontend (React SPA)

React + Vite + Zustand state, Leaflet/OSM maps, Recharts. Routes are gated by SSO role: admins get the dispatcher console, others the self-service portal. A public /track route renders outside the auth gate. Polls the API every few seconds for near-real-time board updates and animates units client-side.

6.2 Core API Lambda

Resolves the principal (API key or JWT), authorises by scope/role, validates input, and routes to handlers: reference reads, fleet, ops, emergency create/override/route, policy upload, tracking, and Power BI token. The dispatch engine is shared across create, mass-casualty and queue-drain paths.

6.3 Dispatch algorithm

- Resolve pickup to coordinates (location ref or raw lat/lng).
- Rank zones by proximity; query the nearest idle unit of the required type via a GSI.
- Medical: choose a hospital matching the case specialty (Critical prefers higher capability, then distance).
- Compute road ETA (OSRM × traffic); persist record; mark unit/driver busy.
- If no unit: QUEUED; a server-side sweep auto-dispatches as units free and completes trips past ETA.

6.4 AI agents

The voice agent uses Bedrock to convert speech/text into structured dispatch slots, confirming before acting. The policy-sync agent uses Bedrock document understanding to extract operational parameters from a PDF and write them to the core Lambda's configuration.

7. Security model

- Authentication: Cognito JWT (RS256) verified against pool JWKS; scoped API keys for servers.
- Authorization: admin via cognito:groups; API-key scopes restrict POST resources; users limited to their own records.
- Transport: HTTPS everywhere; S3 private behind CloudFront OAC.

- Hardening: locked CORS (allow-list origins), API Gateway throttling, strict input validation, generic error messages, no PII in logs.
- Tracking links carry an unguessable per-incident token; no secrets shipped to the browser.

8. Scalability & performance

Every tier scales on demand. Lambda scales horizontally with concurrent requests; DynamoDB on-demand absorbs spikes without capacity planning; CloudFront caches static assets at the edge. The dispatch decision is pure compute over a small in-memory reference cache (warm-invocation reuse). ETA enrichment calls OSRM with per-invocation route caching so mass-casualty fan-out reuses identical routes. API Gateway throttling (10 rps, burst 20) protects downstreams and can be raised per environment.

9. Resilience & failure modes

If this fails…	Behaviour
OSRM unreachable	ETA falls back to straight-line ÷ policy speed × traffic; dispatch still succeeds.
SES/SNS error	Notification is logged and skipped; dispatch is never blocked.
Bedrock unavailable	Voice/policy features degrade; core dispatch (portal/API) is unaffected.
No unit available	Request is QUEUED and auto-dispatched later by the sweep.
Browser/console offline	Server-side sweep still completes trips and drains the queue.
Power BI token error	Dashboard shows a clear message; rest of the app works.

10. Cost model (indicative)

All core services are pay-per-use with no idle cost. Indicative monthly ranges:

Service	Demo scale	Production (township)
Lambda + API Gateway	~Free tier	Low tens of USD
DynamoDB (on-demand)	~Free tier	Low tens of USD
CloudFront + S3	< $1	Single-digit USD
Cognito	Free tier (MAU)	Scales with active users
Bedrock (Nova Lite)	Pennies per call	Usage-based
SES / SNS	< $1	Usage-based (SMS per-message)
Power BI	Free dev embed	Capacity (A/F SKU) for prod

> Power BI is the main step-change cost at production scale: login-free embedding needs a paid capacity (A/F SKU). Everything else remains pay-per-use.